

VibeOS: Lazy Backend-Owned Generation for an Imagined Local Operating System

ZisIsNotZis

Independent developer and researcher

github.com/ZisIsNotZis/vibeos

September 2026

Abstract

VibeOS is a browser-hosted prototype for software and world nodes that may not exist when a user requests them. Its central design is lazy backend-owned generation: the runtime represents a missing surface as an operation, stages a scoped work order, asks an external worker for a structured artifact, validates the candidate, publishes the candidate with rollback-on-load-failure behavior, and exposes the resulting surface. This paper documents the design and the evidence currently present in the repository. The contribution is an implementation architecture and an explicit evidence boundary, not a claim that generated software is generally safe, reliable, or superior to conventional applications. In the inspected source snapshot, 75 server tests pass, including tests for staging, resource-policy selection, structured handoff, artifact validation, rollback, state isolation, and generic runtime behavior. Provider-backed generation quality, latency, and security under adversarial workloads are not established by this artifact. We release a compact source-level account, claim ledger, and reproduction instructions so that stronger claims can be tested rather than inferred.

1 Introduction

Software systems normally assume that an application is installed before its interface is opened. VibeOS explores the inverse interaction: a user addresses a node in a local world tree, and an unavailable surface can be prepared on demand. The system is motivated by local, persistent, extensible software rather than by a general-purpose cloud operating system. It uses a browser for presentation and a backend runtime for durable world state and generation orchestration.

The design combines four ideas. First, applications are replaceable world content behind a generic operating-system boundary. Second, a cache miss is a typed operation rather than an implicit promise. Third, generation is performed in a staged worker workspace with a structured result contract. Fourth, a candidate is published only after source-level checks and can be rolled back if post-publication loading fails. These ideas are related to retrieval and tool-using agents, but VibeOS treats the generated artifact and its lifecycle as the primary unit of delivery.

This paper makes three deliberately narrow contributions: (1) a precise description of the runtime and generation seams implemented in VibeOS; (2) a claim ledger that separates source-backed observations from proposals and unverified hypotheses; and (3) a reproducibility path based on the repository, deterministic tests, and a pinned source snapshot. We do not report an accuracy benchmark, a latency benchmark, or a security proof.

2 System Model

Let W be a durable world tree, R the runtime state, and i a user intent. A surface request is resolved by the following transition:

$$(R, i, W) \rightarrow \begin{cases} (R', \text{ready}, W) & \text{if the requested surface is cached,} \\ (R', \text{pending}, W) \rightarrow (R'', \text{ready}, W') & \text{after validated publication,} \\ (R'', \text{failed}, W) & \text{if generation or validation fails.} \end{cases} \quad (1)$$

The world tree is therefore both content storage and a cache, while mutable application data is kept separately from replaceable source content. This separation is important: replacing an artifact must not silently erase its state.

The core boundary follows a capability-independent rule. The runtime owns addressing, windows, focus, persistence, transport, validation, generation, and diagnostics. Generated content owns its visual design, routes, controls, state schema, and domain behavior. A core branch keyed to a particular application name would violate this boundary and is not part of the intended model.

3 Lazy Generation Harness

3.1 Work orders and staging

For a missing application surface, the harness creates a temporary job below `world/.jobs/`. The job contains `input/`, `framework/`, `output/app/`, and `evidence/`. The work order records a version, operation identifier, capability, original intent, context, acceptance criteria, worker profile, resource policy, and explicit references. An existing application source tree is copied into the output tree without its mutable `data/` directory.

The worker receives an ephemeral invocation, a job working directory, a workspace-write sandbox, a JSON output schema, and a designated result file. Search and resource access are explicit policy metadata; the implemented dimensions are *knowledge*, *assets*, *code*, and *packages*, each with an `off`, `allowed`, or `recommended` level. The policy selects worker behavior and search flags; it is not complete host-side enforcement.

3.2 Structured handoff

The worker result is not inferred from a textual sentinel. A valid result has status `ready` or `needs_input`, a summary, and optional JSON-text state-patch values. Waiting for input cannot be combined with state patches. The AI-command and answer paths decode and apply patches through the revisioned application-state module; surface-generation results expose the structured handoff but do not replay the original control action. A question is emitted as a typed runtime event and may resume the same staged worker through its question identifier.

3.3 Validation and publication

Before publication, the harness rejects symbolic links, non-regular files, multiple hard links, excessive file counts, excessive sizes, and excessive directory depth. The artifact verifier checks local HTML and CSS references and, at non-fast effort, rejects visible buttons without detectable behavior. Publication copies the candidate to an incoming sibling, preserves existing mutable data, swaps the source subtree, invokes an optional post-publication validator, and restores the previous subtree when that callback fails. This is rollback-on-tested-failure, not a crash-safe transaction. The

checks do not establish semantic correctness, browser isolation, provenance, or absence of malicious behavior.

4 Runtime and Bridge

The runtime opens or navigates a window before ensuring its surface. It reuses a ready surface, coalesces concurrent requests for the same app-and-route key, and emits generation and completion events. A generated frame receives a channel-bound bridge for state, storage, notifications, navigation, dispatch, context menus, commands, and AI commands. Bridge dispatch rejects an intent that names another application, and process execution is limited to an allowlist of interpreters with an app-owned working directory and bounded arguments and time.

Application state is namespaced by application identifier and persisted as revisioned JSON. A bridge state write or AI state patch is applied through the same state boundary; stale revisions and invalid patch operations are rejected. This gives generated content a generic interface without granting it unrestricted control of the host runtime.

5 Implementation and Evaluation

Table 1 records the current evidence boundary. Tests are deterministic unit or integration tests unless explicitly marked otherwise. The source snapshot is VibeOS version 0.1.0 at repository revision `90b23f277a52c1862a764a97a9ddd99c8630e4f4`. The reported command ran on 2026-09-01 with Node.js 24.18.1 and npm; it compiled the server TypeScript and executed Node’s built-in test runner.

Table 1: Source-linked evidence available for this paper.

Claim area	Evidence in the repository
Runtime	<code>apps/server/src/runtime.test.ts</code> ; generic windows, cache reuse, persistence, bridge calls, questions, and concurrent request coalescing.
Harness	<code>apps/server/src/generation-harness.test.ts</code> ; staging, profiles, policies, containment arguments, structured results, patches, and rollback.
Artifact checks	<code>apps/server/src/artifact-verifier.test.ts</code> ; broken assets and dead controls are rejected; wired controls are accepted.
World contract	<code>apps/server/src/world-contract.test.ts</code> and <code>world-loader.test.ts</code> ; target containment and endpoint loading.
Observed run	<code>npm test</code> ; TypeScript build plus 75 server tests passed on 2026-09-01.

The evidence demonstrates implemented code paths and test expectations. It does not demonstrate that an arbitrary external model can produce a usable artifact, that the browser UI is free of visual defects, or that a production deployment is safe. No benchmark dataset, provider trace, generated-job corpus, human evaluation, or ablation is included with this package. Browser E2E was not counted as a result: the current Playwright setup has an isolation defect and its last exploratory run reported 19 passed, 6 failed, and 3 skipped while mutating a tracked app-state fixture.

6 Threat Model and Security Boundary

The design assumes that generated content and the external worker may be buggy or adversarial. The intended boundary is layered: jobs are staged below the world root; publication validates filesystem shape; generated frames communicate through a channel-bound bridge; state mutations are namespaced and revision-checked; and process execution uses an interpreter allowlist, bounded arguments, and an app-owned working directory. These mechanisms reduce accidental cross-app writes and malformed artifacts in the tested paths.

They are not a security proof. The current implementation does not claim complete OS/container resource isolation, network denial, provenance verification, malware detection, browser-origin isolation, or protection against every denial-of-service workload. A hostile worker, compromised dependency, malicious generated HTML, or crash at the filesystem-swap boundary requires adversarial evaluation before deployment. The explicit resource policy is therefore an authorization and orchestration seam, not evidence of enforcement.

7 Qualitative Case Study

The repository contains seeded world nodes including games, editors, a browser, and an assistant. A representative runtime path requests an unavailable app surface, opens its window, creates a job containing the intent and inherited memory, runs the worker adapter, validates the candidate tree and generated-world contract, publishes the app source while retaining mutable data, and emits a ready surface or a typed failure/question. The tests exercise this lifecycle with deterministic adapters; they establish the control flow but do not claim that a live provider produced the seeded artifacts. Screenshots in `docs/screenshots/` document the existing browser shell, not a controlled generation-quality experiment.

8 Artifact Availability and Reproducibility

The source package, claim ledger, and bibliography are in `papers/`; the repository is available at <https://github.com/ZisIsNotZis/vibeos>. From its root, run `npm install` and `npm test`. The latter builds the server workspace and runs `node -test dist/*.test.js`. Live generation additionally requires the configured external worker/provider and is not needed for the reported 75-test result. The exact source evidence commit is `90b23f277a52c1862a764a97a9ddd99c8630e4f4`; the paper package is committed separately and identifies that evidence snapshot in `CLAIMS.md`. The paper can be compiled with Tectonic using `tectonic main.tex`.

9 Related Work

Retrieval-augmented generation grounds language-model output in external documents [3]; tool-using agent work emphasizes interleaving reasoning and actions [4]. VibeOS differs in scope: it treats a generated, locally published application surface as a durable cache entry, and it specifies a handoff and publication lifecycle around that entry. Browser isolation and sandboxing are established security research topics [1]; the present artifact uses process and path restrictions but does not claim to reproduce a browser or container security boundary. JSON Patch provides a standard vocabulary for revisioned state mutations [2], which VibeOS uses at its runtime seam.

10 Conclusion

VibeOS shows one implementable pattern for imagined local software: make absence explicit, stage backend-owned work, require a structured handoff, validate before publication, preserve mutable state, and resume the original intent. The current evidence supports this as a prototype architecture and testable implementation. It does not support broad claims about generated-software quality or safety. The next useful experiments are provider-independent generation benchmarks, browser-level verification, adversarial containment tests, provenance accounting, and recovery measurements with published fixtures.

References

- [1] Adam Barth, Collin Jackson, and Charles Reis. The security architecture of the chromium browser. In *USENIX Workshop on Hot Topics in Security*, 2008.
- [2] Paul Bryan and Mark Nottingham. Javascript object notation (json) patch. RFC 6902, 2013.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33, 2020.
- [4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.